The Purpose of Networking Models

The TCP/IP model is a framework used to group networking protocols into **layers** based on their specific functions. It serves as a map of "who does what" in a network, allowing engineers to understand how different protocols, like IP, TCP, and HTTP, work together as a team (often called a **network stack**).

Standards and History

Before standardized models, vendors like IBM and Apple used **proprietary protocols**, which meant devices from different manufacturers often could not communicate.

- **Standards:** Today, networks use **vendor-neutral standards** so that any device (e.g., an Android phone) can communicate with any other device (e.g., a Windows PC).
- **Key Organizations:** Standards are maintained by independent groups like the **IEEE** (responsible for Ethernet and Wi-Fi) and the **IETF** (responsible for TCP, IP, and HTTP via documents called **RFCs**).
- **Origins:** Modern networking began with **ARPANET** in 1969. Vint Cerf and Bob Kahn developed TCP in 1974, which was later split into **TCP** and **IP**. On January 1st, 1983, ARPANET officially switched to the TCP/IP suite.

The 5-Layer TCP/IP Model

The video presents a five-layer model to explain how data moves from a client to a server:

1. **Physical Layer (Layer 1):** Responsible for sending and receiving **bits** as electrical, optical, or radio signals over media like copper UTP cables, fiber-optics, or Wi-Fi.
2. **Local Network Layer (Layer 2):** Also known as the **Data Link** layer. It provides **hop-to-hop** delivery within a local area network (LAN) using **MAC addresses** and switches. Key protocols include Ethernet and Wi-Fi.
3. **Internet Layer (Layer 3):** Also known as the **Network** layer. It provides **end-to-end** delivery between hosts across different networks using **IP addresses** and routers. Protocols include IPv4, IPv6, and ICMP.
4. **Transport Layer (Layer 4):** Provides communication between specific **application processes** (e.g., a web browser vs. a file transfer) using **port numbers**. Key protocols are **TCP** and **UDP**.
5. **Application Layer (Layer 5/7):** The interface where network communication meets software. It defines how data is formatted and interpreted by specific programs, such as **HTTP/HTTPS** for web browsing or **FTP** for files.

Data Flow: Encapsulation and Decapsulation

Communication involves moving data through these layers:

- **Encapsulation:** As data moves **down** the stack on the sending device, each layer adds a **header** (and sometimes a trailer) containing necessary information like IP addresses or port numbers.

- **Decapsulation:** When the receiving device gets the bits, it moves the data **up** the stack, removing and examining the headers at each layer until only the original application data remains.

Message Terminology (PDUs)

At different stages of the process, the data unit has a specific name, known as a **Protocol Data Unit (PDU)**:

- **Segment:** Data + Layer 4 TCP header.
- **Datagram:** Data + Layer 4 UDP header.
- **Packet:** Layer 4 PDU + Layer 3 IP header.
- **Frame:** Layer 3 PDU + Layer 2 header and trailer. The data inside a PDU (excluding that layer's specific header/trailer) is called the **payload**.

Layer Interactions

- **Adjacent-Layer Interaction:** Each layer provides a service to the layer above it and uses the services of the layer below it (e.g., Layer 3 relies on Layer 2 to deliver a packet to the next hop).
- **Same-Layer Interaction:** Each layer on the source host "communicates" with the equivalent layer on the destination host (e.g., the Application layer on a PC sends data to the Application layer on a server).

Model Comparisons

While TCP/IP is the practical standard, the **OSI (Open Systems Interconnection) model** is a 7-layer reference model developed by the ISO in the late 1970s. Although the OSI protocols themselves never gained widespread use over TCP/IP, the OSI layer names and numbers (like calling the Application layer "Layer 7") are still used as a common language in the industry. One of the greatest benefits of these layered models is **modularity**—you can change a protocol at one layer (like switching from a wired to a wireless connection) without needing to redesign the entire network stack.